

DSA Problem-Solving Strategies — How to Identify Patterns & Choose the Right Approach

1. The General Problem-Solving Framework (Do This Every Time)

Before jumping into code, follow this 6-step framework (known as **U-M-P-I-R-E**):

Step	What to Do
U — Understand	Read the problem twice. Identify input, output, and constraints.
M — Match	Match it to a known pattern (see Section 2 & 3 below).
P — Plan	Write pseudocode / dry-run on paper with a small example.
I — Implement	Code it, using clear variable names.
R — Review	Trace through edge cases (empty input, 1 element, duplicates, negative numbers).
E — Evaluate	State Time & Space Complexity. Can it be optimized?

Golden Rule: Never start coding until you can explain your approach out loud in one or two sentences.

2. How to “Read” a Problem — Extracting Clues

Every DSA problem gives you hidden clues. Train yourself to scan for these **4 things** first:

A. Look at the Input Data Structure

Input Given	Likely Pattern Area
Sorted array	Binary Search, Two Pointers
Unsorted array	Sorting, Hashing, Heap
Array of intervals	Merge Intervals
Linked List	Two Pointers (Fast/Slow), Reversal
Tree	DFS / BFS / Recursion
Graph / grid	BFS / DFS / Union-Find / Dijkstra
String	Sliding Window, Two Pointers, Trie
Range of numbers 1 to n	Cyclic Sort

Matrix	DFS/BFS (grid traversal), Dynamic Programming
--------	---

B. Look at the Constraints (n = input size) — This tells you the expected Time Complexity!

Constraint (n)	Expected Complexity	Hints At
$n \leq 10-12$	$O(2^n)$, $O(n!)$	Brute force / Backtracking / Permutations
$n \leq 20-25$	$O(2^n)$	Bitmask DP
$n \leq 500$	$O(n^3)$	Brute force DP, Floyd-Warshall
$n \leq 5,000$	$O(n^2)$	Nested loops, DP on pairs
$n \leq 10^6$	$O(n \log n)$	Sorting, Heap, Binary Search, Divide & Conquer
$n \leq 10^8$	$O(n)$ or $O(\log n)$	Two Pointers, Sliding Window, Hashing, Greedy

Tip: If n is large (10^5 – 10^7 +), an $O(n^2)$ brute force will **Time Limit Exceed (TLE)** — you must optimize.

C. Look at the Question Being Asked (Keyword Signals)

Keyword in Problem	Likely Pattern
“contiguous subarray/substring”	Sliding Window
“pair with sum”, “sorted array”	Two Pointers
“kth largest/smallest”, “top k”	Heap (Priority Queue)
“all combinations/permutations/subsets”	Backtracking
“shortest path”, “minimum steps”	BFS (unweighted) / Dijkstra (weighted)
“connected components”, “islands”	DFS / BFS / Union-Find
“overlapping intervals”, “meetings/schedule”	Merge Intervals
“next greater/smaller element”	Monotonic Stack
“in range 1 to n, find missing/duplicate”	Cyclic Sort
“maximize/minimize”, “optimal”, “count ways”	Dynamic Programming
“prefix”, “word search”, “autocomplete”	Trie
“cycle detection” (linked list)	Fast & Slow Pointers
“merge k sorted lists/arrays”	K-way Merge (Heap)
“topological order”, “prerequisites”	Topological Sort (Graph)
“local vs global choice, no backtrack”	Greedy

D. Look at What Needs to Be Returned

- A single number (count/min/max) → often DP, Greedy, or Math
- All valid answers → Backtracking
- Boolean (yes/no possible) → DP or Graph reachability
- Modified structure (reversed list, sorted array) → In-place manipulation

3. The 15 Core Patterns (Master These and You Can Solve 80% of DSA Problems)

Pattern 1: Two Pointers

When: Sorted array/string, or need to compare pairs from both ends. **Idea:** Use two indices moving toward each other or in the same direction.

```
def two_sum_sorted(arr, target):
    left, right = 0, len(arr) - 1
    while left < right:
        s = arr[left] + arr[right]
        if s == target:
            return [left, right]
        elif s < target:
            left += 1
        else:
            right -= 1
    return [-1, -1]
```

Time: O(n) | **Space:** O(1) **Practice:** Two Sum II, Container With Most Water, 3Sum, Valid Palindrome

Pattern 2: Sliding Window

When: “Contiguous subarray/substring” satisfying some condition (max/min length, sum, distinct chars). **Idea:** Maintain a window [left, right], expand right, shrink left when condition breaks.

```
def max_sum_subarray_k(arr, k):
    window_sum = sum(arr[:k])
    max_sum = window_sum
    for i in range(k, len(arr)):
        window_sum += arr[i] - arr[i - k]
        max_sum = max(max_sum, window_sum)
    return max_sum
```

Time: O(n) | **Space:** O(1) (or O(k) with hashmap) **Practice:** Longest Substring Without Repeating Characters, Minimum Window Substring, Max Sum Subarray of Size K

Pattern 3: Fast & Slow Pointers (Floyd’s Cycle Detection)

When: Linked list problems — cycle detection, finding middle, palindrome check. **Idea:** Slow pointer moves 1 step, fast pointer moves 2 steps.

```
def has_cycle(head):
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
    if slow == fast:
        return True
    return False
```

Time: O(n) | **Space:** O(1) **Practice:** Linked List Cycle, Middle of Linked List, Happy Number

Pattern 4: Merge Intervals

When: Problems involving overlapping ranges, scheduling, calendars. **Idea:** Sort intervals by start time, then merge overlapping ones.

```
def merge_intervals(intervals):
    intervals.sort(key=lambda x: x[0])
    merged = [intervals[0]]
    for start, end in intervals[1:]:
        if start <= merged[-1][1]:
            merged[-1][1] = max(merged[-1][1], end)
        else:
            merged.append([start, end])
    return merged
```

Time: $O(n \log n)$ | **Space:** $O(n)$ **Practice:** Merge Intervals, Insert Interval, Meeting Rooms

Pattern 5: Cyclic Sort

When: Array contains numbers in range 1 to n (or 0 to n-1), find missing/duplicate. **Idea:** Place each number at its correct index ($arr[i] = i+1$) by swapping.

```
def find_missing(arr):
    i = 0
    n = len(arr)
    while i < n:
        correct = arr[i] - 1
        if arr[i] < n and arr[i] != arr[correct]:
            arr[i], arr[correct] = arr[correct], arr[i]
        else:
            i += 1
    for i in range(n):
        if arr[i] != i + 1:
            return i + 1
    return n + 1
```

Time: $O(n)$ | **Space:** $O(1)$ **Practice:** Find Missing Number, Find All Duplicates, First Missing Positive

Pattern 6: In-Place Linked List Reversal

When: Reverse a linked list (fully or in groups) without extra space. **Idea:** Track `prev`, `curr`, `next` pointers and reverse links one by one.

```
def reverse_list(head):
    prev = None
    curr = head
    while curr:
        nxt = curr.next
        curr.next = prev
        prev = curr
        curr = nxt
    return prev
```

Time: $O(n)$ | **Space:** $O(1)$ **Practice:** Reverse Linked List, Reverse in K-Groups, Swap Pairs

Pattern 7: Tree BFS / DFS

When: Any tree traversal — level order, path sum, tree height, ancestors. **Idea:** BFS uses a queue (level by level); DFS uses recursion/stack (depth first).

```
from collections import deque

def bfs(root):
    if not root: return []
    result, queue = [], deque([root])
    while queue:
        level = []
        for _ in range(len(queue)):
            node = queue.popleft()
            level.append(node.val)
            if node.left: queue.append(node.left)
            if node.right: queue.append(node.right)
        result.append(level)
    return result

def dfs(node):
    if not node: return
    print(node.val)           # Preorder
    dfs(node.left)
    dfs(node.right)
```

Time: $O(n)$ | **Space:** $O(n)$ **Practice:** Level Order Traversal, Max Depth, Path Sum, Diameter of Tree

Pattern 8: Graph BFS / DFS

When: Grids, islands, connected components, shortest path (unweighted). **Idea:** Same as tree traversal but track `visited` set (graphs can have cycles).

```
def num_islands(grid):
    if not grid: return 0
    rows, cols = len(grid), len(grid[0])
    visited = set()

    def dfs(r, c):
        if (r < 0 or r >= rows or c < 0 or c >= cols or
            grid[r][c] == '0' or (r, c) in visited):
            return
        visited.add((r, c))
        dfs(r+1, c); dfs(r-1, c); dfs(r, c+1); dfs(r, c-1)

    count = 0
    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == '1' and (r, c) not in visited:
                dfs(r, c)
                count += 1
    return count
```

Time: $O(V+E)$ | **Space:** $O(V)$ **Practice:** Number of Islands, Clone Graph, Word Ladder, Rotting Oranges

Pattern 9: Two Heaps

When: Need running median, or split data into two halves (smaller/larger). **Idea:** Max-heap for smaller half, min-heap for larger half. **Time:** $O(\log n)$ per insert | **Space:** $O(n)$ **Practice:** Find Median

from Data Stream, Sliding Window Median

Pattern 10: Top K Elements (Heap)

When: “Kth largest/smallest”, “top K frequent”. **Idea:** Maintain a heap of size K.

```
import heapq

def top_k_frequent(nums, k):
    from collections import Counter
    count = Counter(nums)
    return heapq.nlargest(k, count.keys(), key=count.get)
```

Time: $O(n \log k)$ | **Space:** $O(k)$ **Practice:** Kth Largest Element, Top K Frequent Elements, K Closest Points

Pattern 11: Subsets / Backtracking

When: “All combinations/permutations/subsets”, constraint satisfaction (N-Queens, Sudoku). **Idea:** Build a solution incrementally; backtrack when constraint violated.

```
def subsets(nums):
    result = []
    def backtrack(start, path):
        result.append(path[:])
        for i in range(start, len(nums)):
            path.append(nums[i])
            backtrack(i + 1, path)
            path.pop()          # undo the choice
    backtrack(0, [])
    return result
```

Time: $O(2^n)$ | **Space:** $O(n)$ **Practice:** Subsets, Permutations, Combination Sum, N-Queens, Word Search

Pattern 12: Modified Binary Search

When: Sorted (or rotated sorted) array, or “search space” reducible by half (search on answer). **Idea:** Standard binary search, adapted with extra conditions.

```
def search_rotated(arr, target):
    left, right = 0, len(arr) - 1
    while left <= right:
        mid = (left + right) // 2
        if arr[mid] == target:
            return mid
        if arr[left] <= arr[mid]:          # left half sorted
            if arr[left] <= target < arr[mid]:
                right = mid - 1
            else:
                left = mid + 1
        else:                             # right half sorted
            if arr[mid] < target <= arr[right]:
                left = mid + 1
            else:
                right = mid - 1
```

```
return -1
```

Time: $O(\log n)$ | **Space:** $O(1)$ **Practice:** Search in Rotated Sorted Array, Find Peak Element, Koko Eating Bananas (binary search on answer)

Pattern 13: Dynamic Programming (DP)

When: “Maximize/minimize”, “count number of ways”, overlapping subproblems + optimal substructure. Also a giveaway: brute force recursion recomputes the same states. **Idea:** Break into subproblems, store results (memoization) or build bottom-up (tabulation).

```
# 0/1 Knapsack - bottom-up
def knapsack(weights, values, capacity):
    n = len(weights)
    dp = [[0] * (capacity + 1) for _ in range(n + 1)]
    for i in range(1, n + 1):
        for w in range(capacity + 1):
            dp[i][w] = dp[i-1][w]
            if weights[i-1] <= w:
                dp[i][w] = max(dp[i][w], dp[i-1][w-weights[i-1]] + values[i-1])
    return dp[n][capacity]
```

Common DP sub-types: 0/1 Knapsack, Unbounded Knapsack, Longest Common Subsequence, Longest Increasing Subsequence, Fibonacci-style, Edit Distance, Matrix Path (grid DP) **Time:** Usually $O(n^2)$ or $O(n \cdot \text{capacity})$ | **Space:** $O(n^2)$, reducible to $O(n)$ **Practice:** Coin Change, LCS, LIS, Edit Distance, Unique Paths, House Robber

Pattern 14: Greedy

When: Local optimal choice leads to global optimal (no need to reconsider past choices). Common in interval/scheduling problems. **Idea:** Sort by a key metric, then make the best choice at each step.

```
def max_meetings(intervals):
    intervals.sort(key=lambda x: x[1])  # sort by end time
    count, last_end = 0, float('-inf')
    for start, end in intervals:
        if start >= last_end:
            count += 1
            last_end = end
    return count
```

Time: $O(n \log n)$ | **Space:** $O(1)$ **Practice:** Activity Selection, Jump Game, Gas Station, Fractional Knapsack

Pattern 15: Monotonic Stack / Union-Find / Trie (Specialized Tools)

- **Monotonic Stack:** “Next Greater Element”, stock span, histogram problems → keep stack increasing/decreasing.
- **Union-Find (Disjoint Set):** “Connected components”, “redundant connection”, Kruskal’s MST → union + find with path compression.
- **Trie:** “Prefix search”, “autocomplete”, “word dictionary” → tree of characters.

4. Quick Decision Flowchart

```

START
|
▼
Is it about a LINKED LIST?
├ Yes → Cycle? → Fast/Slow Pointers
│   Reverse? → In-place Reversal
▼ No
Is it a TREE or GRAPH?
├ Yes → Shortest path/level order → BFS
│   Explore all paths/depth → DFS
│   Connectivity → Union-Find
▼ No
Is the ARRAY/STRING SORTED (or can be sorted)?
├ Yes → Searching → Binary Search
│   Pair sum → Two Pointers
│   Overlap ranges → Merge Intervals
▼ No
Does it ask for CONTIGUOUS subarray/substring?
├ Yes → Sliding Window
▼ No
Does it ask for TOP-K / Kth element / running median?
├ Yes → Heap
▼ No
Does it ask for ALL COMBINATIONS/PERMUTATIONS/SUBSETS?
├ Yes → Backtracking
▼ No
Does it ask to MAXIMIZE/MINIMIZE/COUNT WAYS with overlapping subproblems?
├ Yes → Dynamic Programming
▼ No
Can a LOCAL best choice work without reconsideration?
├ Yes → Greedy
▼ No
→ Default to Brute Force first, then optimize using Hashing/Sorting

```

5. Practical Tips to Get Better at Pattern Recognition

1. **Always start with brute force.** Get a working (even slow) solution first — it reveals the structure of the problem and gives you something to optimize from.
2. **Identify repeated/overlapping work.** If your brute force recomputes the same subproblem, that's a strong signal for **DP**.
3. **Check if sorting helps.** Many "hard" problems become easy Two Pointers/Greedy problems after sorting.
4. **Draw it out.** For trees, graphs, and linked lists — literally draw nodes and pointers on paper before coding.
5. **Track state transitions.** For DP, always ask: "What decision am I making at each step, and what does it depend on?"
6. **Use constraints to eliminate patterns.** If $n \leq 20$, don't look for $O(n)$ — the problem probably wants backtracking/bitmask DP.
7. **Practice by pattern, not randomly.** Solve 5-10 problems from ONE pattern before moving to the next — this builds true recognition instead of memorization.
8. **Re-solve problems after a week** without looking at your old solution — this cements the pattern in long-term memory.
9. **Maintain a "pattern journal."** For every problem you solve, write: Pattern used → Trigger keyword → Time Complexity. Review it weekly.
10. **When stuck for 20-30 minutes**, look at just the *hint/tag* (not the solution) — this trains your intuition faster than either giving up immediately or struggling for hours.

6. Suggested Practice Order (Beginner → Advanced)

1. Arrays & Strings (Two Pointers, Sliding Window)
 2. Linked Lists (Fast/Slow, Reversal)
 3. Stacks & Queues (Monotonic Stack)
 4. Binary Search
 5. Trees (BFS/DFS, Recursion)
 6. Backtracking
 7. Heaps (Top K, Two Heaps)
 8. Graphs (BFS/DFS, Union-Find, Topological Sort)
 9. Dynamic Programming (1D → 2D → advanced)
 10. Greedy & Advanced Graphs (Dijkstra, MST)
-

End of DSA Problem-Solving Strategy Guide